INFORME DE LABORATORIO

Autores: *Cristian Florez Grisales, Daniel Eduardo López Rodríguez*

Laboratorio de Electrónica Digital 3
Departamento de Ingeniería Electrónica y de Telecomunicaciones
Universidad de Antioquia

Resumen

A lo largo de la práctica se desarrollaron dos implementaciones de un sistema embebido basado en una Raspberry Pi Pico, con el propósito de realizar una comparación entre dos entornos de desarrollo con diferentes características: Arduino IDE, mediante C/C++, y Thonny, mediante MicroPython. Para ambas implementaciones se establecieron los mismos requisitos funcionales, parámetros de funcionamiento y condiciones de prueba, permitiendo evaluar su comportamiento bajo condiciones equivalentes. Se implementó un sistema interactivo basado en el juego de memoria "PulsoMemoria", en el cual se integraron botones, LED y displays de siete segmentos, además de una máquina de estados encargada de gestionar las diferentes etapas del juego. Se realizaron pruebas individuales de los módulos y pruebas de integración para verificar el cumplimiento de los requisitos establecidos. Los resultados permitieron comprobar que ambas implementaciones cumplen satisfactoriamente con la funcionalidad esperada, presentando diferencias principalmente relacionadas con el proceso de desarrollo y la gestión de los recursos del microcontrolador.

Palabras clave: Raspberry Pi Pico; sistemas embebidos; Arduino IDE; MicroPython; comparación de entornos.

Introducción

Es normal que en el desarrollo de sistemas embebidos existan diferentes entornos de trabajo, cada uno con ventajas y desventajas en relación con los demás. Por esta razón, la presente práctica se enfoca en la comparación de dos entornos de desarrollo: Arduino IDE y Thonny, este último empleado para la programación mediante MicroPython. La comparación se realiza a partir de la implementación de un sistema interactivo basado en el juego de memoria "Simón dice", en el cual se integran entradas digitales, salidas luminosas, visualización numérica y restricciones temporales.

El propósito de la práctica es desarrollar dos implementaciones funcionalmente equivalentes del juego, utilizando el mismo hardware, requisitos y pruebas de aceptación, de manera que sea posible evaluar las diferencias entre ambos entornos bajo condiciones comparables. Para ello, se analizará el comportamiento del sistema durante las diferentes etapas del juego, considerando aspectos como los tiempos de respuesta, la implementación e integración de los módulos y la organización del software.

Se plantea como hipótesis que ambos entornos permiten implementar un sistema con las mismas características funcionales y cumplir los requisitos establecidos; sin embargo, pueden presentarse diferencias en el desempeño temporal, la complejidad del desarrollo y la facilidad de implementación debido a las características propias de cada entorno y del lenguaje utilizado. En particular, se busca establecer experimentalmente las ventajas, desventajas y condiciones de uso de Arduino IDE, basado en C++ (como lenguaje compilado), y Thonny, empleado con MicroPython (como lenguaje interpretado), a partir de las pruebas realizadas sobre ambas implementaciones.

Marco teórico

Arduino IDE y C/C++:

Arduino IDE es un entorno de desarrollo integrado utilizado para escribir, compilar y cargar programas en placas de desarrollo compatibles. En el contexto de la Raspberry Pi Pico, este entorno permite utilizar el ecosistema de Arduino para programar el microcontrolador mediante C/C++, proporcionando acceso a los periféricos y recursos de hardware de la placa. La Raspberry Pi Pico puede ser programada mediante diferentes lenguajes, entre ellos C/C++, MicroPython y otros entornos compatibles, lo que permite seleccionar la alternativa de acuerdo con los requerimientos de la aplicación [1].

El lenguaje C/C++ permite un mayor control sobre los recursos del microcontrolador y resulta adecuado para aplicaciones en las que el desempeño y la eficiencia son relevantes. En el caso de Arduino, el código desarrollado debe ser compilado antes de ser ejecutado en el microcontrolador. De esta manera, Arduino IDE permite desarrollar aplicaciones embebidas con acceso directo a las entradas, salidas, temporizadores y demás periféricos disponibles en el microcontrolador.

Thonny y MicroPython:

Thonny es un entorno de desarrollo orientado principalmente a Python y que incorpora soporte para MicroPython. En el caso de la Raspberry Pi Pico, permite seleccionar el intérprete de MicroPython correspondiente a la placa, escribir y ejecutar programas directamente sobre el microcontrolador. Además, proporciona herramientas como una consola interactiva y un depurador, facilitando la ejecución y comprobación del código durante el desarrollo [1].

MicroPython es una implementación de Python desarrollada específicamente para microcontroladores. Conserva gran parte de la sintaxis característica de Python, pero está adaptada a las restricciones y características propias de estos dispositivos. Esto permite controlar elementos del hardware de la Raspberry Pi Pico mediante una sintaxis de programación de alto nivel, facilitando el desarrollo de aplicaciones embebidas [1].

Una característica importante de este entorno es la posibilidad de interactuar con el microcontrolador mediante la consola interactiva de Thonny. El programa puede escribirse en el editor y ejecutarse directamente en la Raspberry Pi Pico, además de permitir observar mensajes y errores durante la ejecución. Estas características hacen que MicroPython y Thonny sean una alternativa accesible para el desarrollo y depuración de sistemas embebidos [2].

Aunque Arduino IDE y Thonny permiten desarrollar aplicaciones para la Raspberry Pi Pico, presentan diferencias en el proceso de desarrollo. Arduino utiliza C/C++, mientras que Thonny permite trabajar con MicroPython, por lo que existen diferencias tanto en el lenguaje utilizado como en la forma de ejecutar el programa. C/C++ proporciona un mayor control y eficiencia sobre los recursos del microcontrolador, mientras que MicroPython busca facilitar el desarrollo mediante una sintaxis de alto nivel y herramientas orientadas a la interacción y depuración del programa [1].

Estas diferencias hacen posible comparar ambos enfoques mediante una misma aplicación embebida. En la presente práctica, las dos implementaciones utilizarán el mismo hardware, las mismas funcionalidades y los mismos criterios de aceptación. De esta manera, las diferencias observadas en aspectos como el tiempo de respuesta, la complejidad del código, la integración de los módulos y el proceso de desarrollo podrán asociarse a las características de cada entorno y lenguaje, manteniendo constantes las condiciones principales del sistema.

Procedimiento experimental y resultados

Para ambas implementaciones se definió una arquitectura modular compuesta por tres módulos funcionales: **Juego**, **Dispositivos** y **decoder_display**, los cuales son integrados desde un módulo principal denominado **PulsoMemoria**. Esta estructura permite separar la lógica del juego, el control del hardware y la gestión de los displays, manteniendo responsabilidades diferenciadas entre los componentes del sistema.

Juego: encargado de gestionar las reglas y el estado del juego. En este módulo se genera la secuencia aleatoria, se validan las entradas del usuario y se realiza el seguimiento del tiempo, el nivel y las vidas disponibles. Asimismo, se implementa la máquina de estados que determina las diferentes etapas del juego.

Dispositivos: encargado de la interacción con el hardware. En este módulo se inicializan los pines correspondientes a los botones, LED y demás dispositivos de salida, se realizan las lecturas de las entradas del usuario y se implementan las animaciones asociadas a los diferentes estados del juego.

decoder_display: encargado de la decodificación y multiplexación de los displays de siete segmentos, utilizados para mostrar el nivel, las vidas y el tiempo acumulado durante la ejecución del juego.

PulsoMemoria: corresponde al módulo principal, encargado de integrar los módulos anteriores y ejecutar la máquina de estados, Figura **0-1** que determina las diferentes etapas del juego.

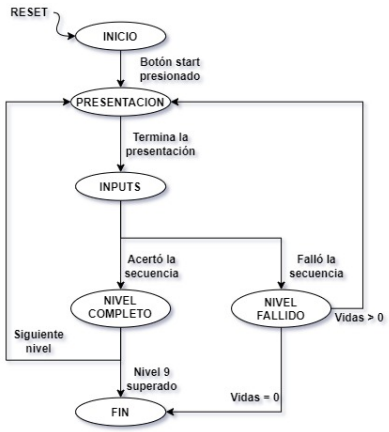


Figura **0-1**: Máquina de estados implementada.

A partir de las responsabilidades definidas para cada módulo, se establecieron pruebas funcionales individuales y pruebas de integración. Las primeras permiten verificar el comportamiento de las funciones asociadas a cada componente de manera independiente, mientras que las segundas permiten comprobar la interacción entre los módulos y el cumplimiento de los requisitos funcionales del sistema completo. Las pruebas fueron aplicadas de manera equivalente a las implementaciones desarrolladas en Arduino IDE y MicroPython, manteniendo las mismas condiciones de hardware y criterios de aceptación. Dichas pruebas se detallan en el la matriz de trazabilidad, Cuadro **0-1**

Discusión de resultados

A partir de las pruebas de verificación realizadas se evaluó el cumplimiento de los requisitos funcionales establecidos para ambas implementaciones. Los resultados obtenidos para cada requisito se presentan a continuación, utilizando las evidencias experimentales correspondientes para verificar el comportamiento del sistema.

RF01: Para la prueba PT01 se obtuvo un resultado satisfactorio, cumpliendo con el comportamiento esperado. La multiplexación de los displays se verificó mediante la impresión en consola, en ambas implementaciones, del display que se encontraba activo y del valor asociado a este. De esta manera, se comprobó que los cuatro displays son activados de manera secuencial y que cada uno presenta la información correspondiente. Las evidencias obtenidas se presentan en las Figuras **0-2** y **0-3**.

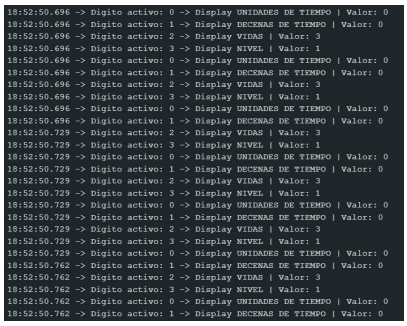


Figura **0-2**: Multiplexación en Arduino IDE.

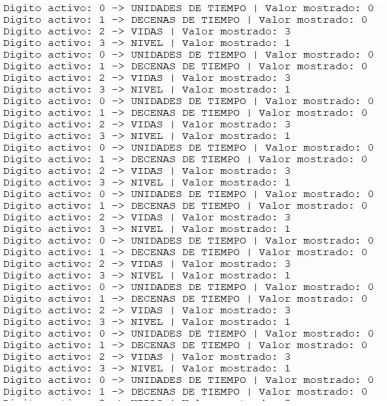


Figura **0-3**: Multiplexación en Thonny.

RF02: La prueba PT02 permitió verificar la lectura de las entradas del usuario. Se evidenció la correcta detección de los eventos asociados a START/RESET y de los botones utilizados para ingresar la secuencia de respuesta. Asimismo, se verificó el comportamiento de antirrebote definido para las entradas, obteniéndose el comportamiento esperado en ambas implementaciones. Como evidencias las Figuras **0-4** y **0-5**

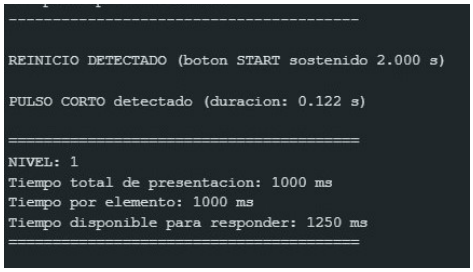


Figura **0-4**: Lectura de botones en Arduino IDE.

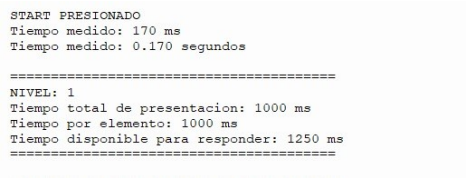


Figura **0-5**: Lectura de botones en Thonny.

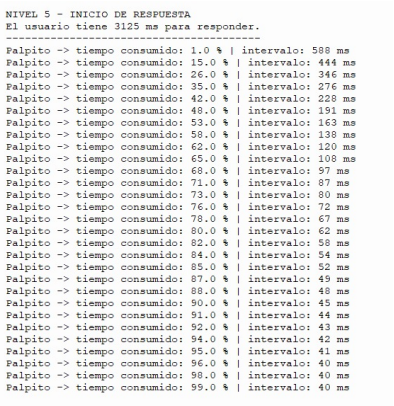


Figura **0-7**: Porcentaje y tiempo de palpito en Thonny.

RF03: La prueba PT03 permitió verificar el comportamiento progresivo del LED asociado al tiempo disponible para responder. Se comprobó que el intervalo de encendido y apagado del LED se modifica de acuerdo con el porcentaje de tiempo restante, produciendo un parpadeo progresivamente más rápido a medida que se aproxima el agotamiento del tiempo establecido. Evidencias en las figuras **0-6** y **0-7**

RF04: La prueba PT04 presentó un resultado satisfactorio, debido a que se verificó el cálculo de los diferentes intervalos temporales utilizados por el sistema. Se evidenciaron los tiempos correspondientes a la presentación de la secuencia, el tiempo disponible para la respuesta y el tiempo empleado por el usuario, comprobándose que estos valores son contabilizados de acuerdo con las condiciones establecidas para el juego. Evidencias en las Figuras **0-8** y **0-9**

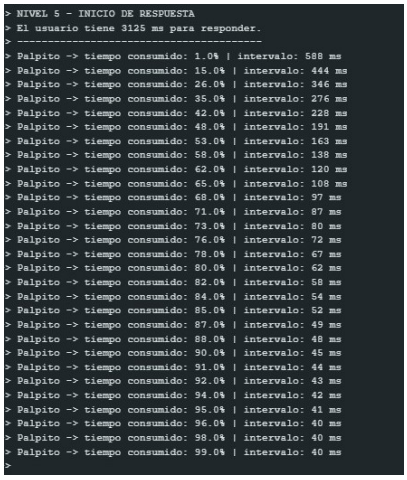


Figura **0-6**: Porcentaje y tiempo de palpito en Arduino IDE.



Figura **0-8**: Tiempos de presentación y respuesta en Arduino IDE.

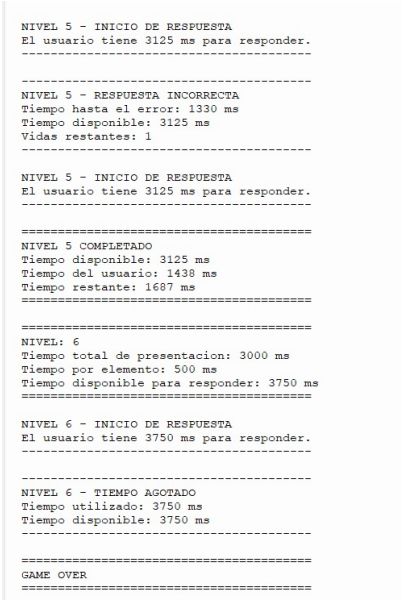


Figura **0-9**: Tiempos de pressentación y respuesta en Thonny.

RF05: La prueba PT05 permitió verificar la integración de los diferentes módulos del sistema. Se comprobó el cumplimiento de las transiciones entre los estados definidos, así como la imposibilidad de ingresar datos durante la etapa de presentación de la secuencia. También se verificó que los valores mostrados de nivel, vidas y tiempo corresponden al estado actual del juego y que el reinicio solamente puede efectuarse mediante una pulsación sostenida del botón RESET. Finalmente, se verificaron las secuencias de indicación correspondientes a respuestas correctas e incorrectas y las condiciones de finalización del juego. Evidencia de todo el sistema en funcionamiento en Video

En términos funcionales, ambas implementaciones cumplieron los requisitos establecidos y presentaron el comportamiento esperado durante las pruebas de verificación. No se observaron diferencias relevantes en la ejecución de las etapas del juego, la lectura de las entradas, la actualización de las salidas ni en las transiciones de la máquina de estados. Asimismo, los tiempos medidos para ambas implementaciones presentaron valores similares y no afectaron el comportamiento perceptible del sistema.

Conclusiones

Es pertinente considerar que cada entorno posee un enfoque diferente, por lo que la implementación del sistema a nivel de programación presenta algunas diferencias. En el entorno Arduino IDE, mediante el uso de C/C++, el código es compilado antes de su ejecución, lo que permite un mayor control sobre los recursos disponibles del microcontrolador y sobre las características específicas del hardware, además de posibilitar el uso de diferentes librerías. Sin embargo, este enfoque requiere un mayor conocimiento de programación y de la arquitectura y funcionamiento del microcontrolador.

Por otro lado, el entorno Thonny, mediante la implementación de MicroPython, presenta un enfoque más amigable para el desarrollador, especialmente para usuarios con menor experiencia en programación de sistemas embebidos, debido a que permite trabajar con un lenguaje de alto nivel y una sintaxis más sencilla. No obstante, esta facilidad de desarrollo implica un mayor uso de recursos computacionales, ya que MicroPython requiere un intérprete que ejecuta el código durante su funcionamiento, además de la memoria necesaria para mantener el entorno de ejecución. Por esta razón, aunque ambos enfoques permiten desarrollar el mismo sistema y cumplir los requisitos funcionales establecidos, presentan diferencias en la forma en que utilizan los recursos del microcontrolador.

En consecuencia, la selección entre ambos entornos depende de las características y necesidades de la aplicación. Arduino IDE y C/C++ pueden resultar más convenientes cuando se requiere un mayor control sobre el hardware, eficiencia en el uso de recursos y un comportamiento temporal más preciso, mientras que Thonny y MicroPython pueden ser más adecuados cuando se prioriza la facilidad de desarrollo, legibilidad del código y rapidez de implementación. Para el sistema desarrollado, ambas implementaciones cumplieron satisfactoriamente los requisitos funcionales establecidos, por lo que las diferencias entre ellas se encuentran principalmente en el proceso de desarrollo y en la forma de gestionar los recursos del microcontrolador.

Bibliografía

[1] Mohamed Fezari, Ali Al Dahoud, and Ahmad Al-dahoud. Introduction of raspberry pi pico in embedded system course: Setup, programming & applications. Technical report, ResearchGate, 12 2024. URL https://www.researchgate.net/publication/387493982_Introduction_of_Raspberry_Pi_Pico_In_Embedded_system_Course_Setup_Programming_Applications.

[2] RaspberryTips. Getting started with c++ on raspberry pi (guide & examples), 2025. URL https://raspberrytips.com/use-c-on-raspberry-pi/.

Cuadro **0-1**: Matriz de trazabilidad de las pruebas de verificación.

ID	Requisito	Módulo	Prueba	Criterio de aceptación
RF01	Multiplexación de displays	decoder_display	PT01: Verificación de la multiplexación de los displays	Cada display se activa de manera independiente durante un intervalo de tiempo y muestra correctamente el valor asociado al nivel, vidas, decenas o unidades del tiempo acumulado.
RF02	Lectura de botones	Dispositivos	PT02: Lectura de botones y START/RESET	Se diferencian correctamente las entradas correspondientes a START/RESET y a los botones de juego, incluyendo el comportamiento de antirrebote.
RF03	Pálpito progresivo del LED de tiempo	Dispositivos	PT03: Verificación de los tiempos de encendido y apagado del LED	La frecuencia de parpadeo aumenta progresivamente a medida que disminuye el tiempo disponible para responder.
RF04	Control temporal	Juego	PT04: Verificación de los tiempos del juego	Los tiempos de presentación, tiempo disponible para responder y tiempo empleado por el usuario son calculados y contabilizados correctamente.
RF05	Integración del sistema	PulsoMemoria	PT05: Ejecución completa del sistema	El sistema cumple las transiciones de estado, reglas del juego, visualización, interacción con el usuario y condiciones de finalización establecidas.